

Métodos: guia completo

Consulta, ação, parâmetros, return vs print, e um método chamando outro.

RESUMO, SE VOCÊ SÓ TEM 1 MINUTO

Método é uma função que mora dentro de uma classe e sempre recebe `self` como primeiro parâmetro. Métodos podem só **ler** dados (consulta), ou **mudar** o objeto (ação). Alguns `print` am um resultado; outros `return` am um valor pra ser usado depois — são coisas diferentes. E um método pode chamar outro método do mesmo objeto, usando `self.outro_metodo()`.

O QUE TEM AQUI

1. O que é um método (recap rápido)
2. Anatomia de um método, pedaço por pedaço
3. Métodos de consulta vs métodos de ação
4. Parâmetros além do `self`
5. `return` vs `print` — a diferença que importa
6. Um método chamando outro método do mesmo objeto
7. Erros comuns (e como consertar)
8. Perguntas frequentes
9. Checklist mental
10. Glossário rápido

1. O que é um método (recap rápido)

Desde o Dia 1, método é a palavra pra "uma ação que a ficha sabe fazer". Um atributo (`nome`, `vida`) guarda um dado. Um método faz alguma coisa com esse dado — mostra na tela, muda o valor, ou calcula algo novo a partir dele.

Todo método vive *dentro* do corpo da classe, é definido com `def`, e sempre recebe `self` como primeiro parâmetro — exatamente como o `__init__`, só que sem o nome reservado. Vocês escolhem o nome do método; `__init__` é o único com nome fixo.

2. Anatomia de um método, pedaço por pedaço

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        print(self.nome, "-", self.vida, "de vida")
```

- `def mostrar_status(self):` — `def` igual a qualquer função; `mostrar_status` é um nome escolhido por vocês (verbos costumam ficar mais claros que substantivos); `self` em primeiro lugar, sempre.
- Dentro do corpo, `self.nome` e `self.vida` acessam os atributos *desse* objeto específico — o mesmo `self` que já apareceu no `__init__`.

Chamando de fora da classe:

```
thoric = Personagem("Thoric", 100, 15)
thoric.mostrar_status()
```

Repara que na chamada **não** se passa `self` — `thoric.mostrar_status()`, sem argumento nenhum. O Python já sabe que `self` é o próprio `thoric`, porque foi ele quem chamou o método.

3. Métodos de consulta vs métodos de ação

Vale a pena notar a diferença entre os dois tipos, mesmo que a sintaxe seja parecida:

CONSULTA — SÓ LÊ

```
def esta_vivo(self):
    return self.vida > 0

def mostrar_status(self):
    print(self.nome, self.vida)
```

AÇÃO — MUDA O OBJETO

```
def receber_dano(self, dano):
    self.vida -= dano

def curar(self, quantidade):
    self.vida += quantidade
```

Um método de consulta pode ser chamado quantas vezes quiser sem "gastar" nada — só olha e informa. Um método de ação muda

`self.algo` permanentemente; chamar duas vezes dá um resultado diferente de chamar uma vez só. Antes de escrever um método novo, vale se perguntar: *ele só olha, ou ele muda alguma coisa?*

4. Parâmetros além do self

Um método pode receber parâmetros extras, exatamente como qualquer função — eles só entram *depois* de `self` na definição:

```
def receber_dano(self, quantidade):
    self.vida = self.vida - quantidade
```

Na hora de chamar, só se passa o que vem depois de `self` — de novo, `self` nunca é digitado na chamada:

```
thoric.receber_dano(30)
```

Por que a definição tem 2 parâmetros mas a chamada só passa 1 valor? Porque o Python preenche `self` sozinho com o objeto que fez a chamada (`thoric`). Só o resto — `quantidade` — precisa ser fornecido explicitamente.

5. return vs print — a diferença que importa

Os dois colocam informação "pra fora" do método, mas de formas bem diferentes, e confundir os dois é um dos erros mais comuns dessa fase:

- `print(...)` só *mostra* algo na tela. O método continua sem devolver nada que possa ser guardado ou reaproveitado.
- `return valor` *devolve* o valor pra quem chamou o método — pra ser guardado numa variável, comparado, somado, usado em outro lugar.

```
class Personagem:
    def mostrar_ataque(self):
        print(self.ataque)          # só mostra na tela

    def calcular_ataque(self):
        return self.ataque          # devolve o valor pra ser usado
```

```
thoric.mostrar_ataque()             # aparece "15" na tela, e só

valor = thoric.calcular_ataque()    # valor agora guarda 15
print(valor * 2)                    # 30 — dá pra usar o valor livremente
```

Um bug silencioso pra ficar de olho: tentar guardar o resultado de um método que só imprime não dá erro nenhum — só guarda algo errado, sem avisar:

```
dano = thoric.mostrar_ataque()
print(dano)                        # None — porque mostrar_ataque nunca teve um return
```

Esse é um dos erros mais difíceis de perceber, porque o Python não reclama — ele só devolve `None` silenciosamente. Se uma variável está vindo `None` sem motivo aparente, o primeiro lugar pra olhar é se o método chamado realmente tem um `return`.

6. Um método chamando outro método do mesmo objeto

Dentro de um método, dá pra chamar outro método do mesmo objeto usando `self.outro_metodo()` — exatamente como se acessa um atributo com `self.atributo`, só que chamando uma ação em vez de ler um valor:

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def esta_vivo(self):
        return self.vida > 0

    def mostrar_status(self):
        if self.esta_vivo():
            print(self.nome, "esta de pe.")
        else:
            print(self.nome, "morreu.")
```

`mostrar_status` não repete a lógica de "vida maior que zero" — ele só pergunta pro próprio objeto, através de `self.esta_vivo()`. Se um

dia a definição de "estar vivo" mudar (por exemplo, incluir um status de "atordoadado"), só precisa mudar dentro de `esta_vivo`, e todo mundo que chama esse método já se beneficia automaticamente.

Isso é o mesmo princípio de reaproveitamento que motivou criar classes lá no Dia 1: em vez de repetir a mesma lógica em vários lugares, ela mora num único método, e todo o resto só chama.

7. Erros comuns (e como consertar)

Esquecer os parênteses na hora de chamar

CÓDIGO COM ERRO

```
thoric.mostrar_status
```

O QUE ACONTECE

```
(nada aparece na tela — nenhum erro é lançado)
```

CORREÇÃO

```
thoric.mostrar_status()
```

Sem os parênteses, `thoric.mostrar_status` só aponta pro método — não manda ele *rodar*. É outro erro silencioso: nada quebra, mas nada acontece também. Se um método "não fizer nada" quando chamado, o primeiro lugar pra checar são os parênteses no final.

Passar self manualmente na chamada

CÓDIGO COM ERRO

```
thoric.receber_dano(thoric, 30)
```

O QUE O PYTHON MOSTRA

```
TypeError: receber_dano() takes 2 positional arguments
but 3 were given
```

CORREÇÃO

```
thoric.receber_dano(30)
```

`self` já é preenchido automaticamente pelo `thoric`. antes do ponto — passar de novo manualmente sobra um argumento.

Esquecer o return e usar o resultado mesmo assim

CÓDIGO COM ERRO

```
def calcular_ataque(self):
    self.ataque * 2          # calcula, mas nao devolve

dano = thoric.calcular_ataque()
print(dano)
```

O QUE ACONTECE

```
None
```

CORREÇÃO

```
def calcular_ataque(self):
    return self.ataque * 2
```

Definir um método fora da classe, por erro de indentação

CÓDIGO COM ERRO

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida

def mostrar_status(self):
    print(self.nome)
```

O QUE O PYTHON MOSTRA

```
NameError: name 'self' is not defined
```

CORREÇÃO

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida

    def mostrar_status(self):
        print(self.nome)
```

`mostrar_status` precisa da mesma indentação de `__init__` pra continuar dentro do corpo da classe. Um nível de indentação a menos, e ele vira uma função solta, sem relação nenhuma com `Personagem` — o mesmo tipo de erro de indentação da lição do Dia 1, só que agora dentro de uma classe.

8. Perguntas frequentes

Quantos métodos uma classe pode ter?

Quantos forem necessários. Não tem limite prático.

O nome do método pode ser qualquer um?

Sim, seguindo as mesmas regras de nomes de variáveis do Python. Por convenção, use verbos (`atacar`, `calcular_dano`, `mostrar_status`) — deixa mais claro que é uma ação, não um dado.

Um método pode chamar um método de OUTRO objeto, não só do mesmo?

Sim — por exemplo, `inimigo.receber_dano(thoric.ataque)` usa um atributo de `thoric` como argumento pra um método de `inimigo`. Isso é o assunto do próximo guia, sobre organizar várias classes num só `main.py`.

Um método sem return devolve mesmo assim alguma coisa?

Sim — devolve `None` automaticamente, sempre. É por isso que guardar o resultado de um método sem `return` não dá erro, só guarda `None` silenciosamente (seção 5).

9. Checklist mental

Antes de considerar um método pronto, confira:

- Começa com `def nome_do_metodo(self, ...):`, com `self` em primeiro lugar?
- Se ele muda o objeto, tem pelo menos um `self.atributo = ...` dentro?
- Se o valor precisa ser usado depois (guardado, comparado, somado), ele usa `return`, não só `print`?
- Se ele chama outro método do mesmo objeto, usa `self.outro_metodo()`, com parênteses?
- Na hora de chamar de fora, tem os parênteses no final: `objeto.metodo()`, mesmo sem argumentos?
- O método está indentado no mesmo nível que os outros métodos da classe?

10. Glossário rápido

método de consulta

método que só lê e informa dados, sem mudar o objeto.

método de ação

método que muda o estado do objeto, geralmente reescrevendo algum `self.atributo`.

return

devolve um valor pra quem chamou o método, pra ser guardado ou reaproveitado.

None

valor especial do Python que significa "nenhum valor" — é o que um método devolve por padrão quando não tem `return`.

estado

o conjunto de valores que um objeto guarda em um dado momento (seus atributos atuais).

Se um método "não funcionar" e nenhum erro aparecer, desconfie primeiro de parênteses esquecidos ou de um return faltando — são os dois erros mais silenciosos desse assunto.